

METODOLOGÍA DE SISTEMAS II

Trabajo Práctico: Flujos de trabajo colaborativo

Parte 1: Estrategias de Branching (Modelos de Ramificación)

1. Análisis de Modelos: Explique las diferencias fundamentales entre **GitFlow** y **Trunk-based Development** considerando los siguientes factores de gestión humana:

- La estructura de comunicación del equipo (jerárquica vs. sincronizada).
- El costo de la "Deuda de Integración" (aislamiento prolongado de ramas).
- La complejidad de los conflictos de fusión (*Merge Hell*).

2. Caso de Estudio: Selección de Estrategia: Una empresa de software médico está desarrollando un sistema crítico que requiere auditorías estrictas y solo lanza actualizaciones cada tres meses. El equipo es grande y jerárquico.

- ¿Qué modelo de ramificación recomendaría y por qué? Justifique su respuesta basándose en el control y la estabilidad.

3. Feature Flags (Interruptores de Código): En equipos que integran cambios diariamente a la rama principal, ¿cómo permiten las **Feature Flags** que un programador suba código de una funcionalidad incompleta sin afectar la experiencia del usuario final?

Parte 2: Procesos de Integración Colaborativa

4. El Pull Request (PR) como Herramienta de Comunicación: Un PR no es solo un comando técnico, es una petición formal. Mencione qué elementos de información son indispensables en una **descripción de PR** para que un compañero pueda entender el cambio sin tener que leer línea por línea el código inicialmente.

5. La Cultura de la Code Review: ¿Cuál es el propósito de una revisión de código más allá de encontrar errores de sintaxis? Explique la importancia de la **transferencia de conocimiento** y el cumplimiento de principios como **DRY** (Don't Repeat Yourself) y **SOLID**.

6. Estrategias de Merge y el Historial: Compare las siguientes tres formas de integrar una rama y explique qué impacto tienen en la estética del grafo de Git:

- **Create a Merge Commit** (Historial verídico).

- **Squash and Merge** (Historial limpio).
- **Rebase and Merge** (Historial lineal).

Parte 3: Estandarización y Documentación del Proyecto

7. Guías de Contribución (CONTRIBUTING.md): Usted es el líder de un proyecto y llega un nuevo integrante. ¿Qué información específica debería encontrar este desarrollador en el archivo CONTRIBUTING.md para empezar a codificar siguiendo las reglas del equipo?

8. Architecture Decision Records (ADRs): Explique la utilidad de un **ADR**. ¿Cómo ayuda este documento a que un desarrollador que se suma al proyecto un año después entienda por qué se tomó una decisión técnica específica en el pasado?

9. Plantillas de Trabajo (Templates): ¿Qué beneficio aporta el uso de plantillas para **Issues** y **Pull Requests** en la organización diaria de un equipo de desarrollo?

Parte 4: Validación de Estándares Locales

10. Git Hooks y la "Primera Línea de Defensa": Explique qué es un **Git Hook** (como Husky) y cómo ayuda a que el equipo mantenga un estándar de código (Linting) antes de que los cambios siquiera lleguen al repositorio compartido. ¿Por qué es importante validar el código en la máquina del desarrollador?